VeriChain - Document Verification System with Ethereum Smart Contracts

Submitted in partial fulfillment of the requirements of the degree

BACHELOR OF ENGINEERING IN COMPUTER ENGINEERING

By

Nidhi Sinha (39)

Shruti Patel(42)

Susmita Sahu (53)

Under Guidance of

Prof. Selvamurugan Kasi



Department of Computer Engineering

Smt. Indira Gandhi College of Engineering

Affiliated to University of Mumbai

(2026-2027)

CERTIFICATE

This is to certify that the Mini Project entitled **"VeriChain - Document Verification System with Ethereum Smart Contracts"** is a bonafide work of **Nidhi Sinha (39), Shruti Patel (42) & Susmita Sahu (53)** submitted to the University of Mumbai in partial fulfillment of the requirements for the award of the degree of **"Bachelor of Engineering" in "Computer Engineering".**

(Prof. **Selvamurugan Kasi**)
Guide

(Dr. K .T .Patil)
Head of Department

(Dr. Sunil Chavan)
Principal

Mini Project Approval

This Mini Project entitled **"VeriChain - Document Verification System with Ethereum Smart Contracts"** By **Nidhi Sinha (39), Shruti Patel (42) & Susmita Sahu (53)** is approved for the degree of Bachelor of Engineering in Computer Engineering.

Examiners

1.....

(Internal Examiner Name & Sign)

2.....

(External Examiner Name & Sign)

Date :

Place :

DECLARATION

We declare that this written submission represents our own ideas in our words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Nidhi Sinha

_____

Shruti Patel

_____

Susmita Sahu

_____

INDEX

CONTENTS

Chapter No	Page No
Abstract	**i**
List of Abbreviation	**ii**
List of Figures	**iii**
List Of Tables	**iv**
1.INTRODUCTION	**9**
1.1 Problem Statement	10
1.2 Objective	11
1.3 Scope	12
1.4 Report Organization	13
2.REVIEW OF LITERATURE	**14**
2.1 Literature Survey	15
2.2 Limitation Existing System/Research Gap/Comparative Analysis	15
2.3 Mini Project Contribution	16
3.PROPOSED SYSTEM	**17**
3.1 Introduction	18
3.2 Architecture/Framework	19
3.3 System Design and Algorithm	20
3.4 Detail of Hardware & Software	22
3.5 System Implementation	22
3.6 Experiment and Result	23
3.7 Conclusion and Future Work	27
REFERENCES	**29**
ACKNOWLEDGMENT	**30**

ABSTRACT

Academic certificates, identity proofs and official letters are still verified mostly by hand, which is slow and makes forged documents hard to catch. This report presents an VeriChain - Document Verification System that combines artificial intelligence with the Ethereum blockchain to make document verification fast, tamper-proof and publicly checkable. A user uploads a PDF or image document; the system extracts its text with Tesseract OCR, runs an AI analysis that estimates the document type, confidence and forgery risk, and computes a SHA-256 hash of the file. That hash is then registered on the blockchain through a Solidity smart contract, deployed on a local Ethereum network (Ganache/Hardhat) and accessed from a Django back end using Web3.py.

Any verifier can later upload the same document: the system recomputes the hash and checks it against the smart contract, instantly reporting whether the document is genuine and unaltered or not registered. Test results show that all core features—upload, AI analysis, blockchain registration and verification—work reliably, demonstrating the potential of combining AI and blockchain for trustworthy digital document management.

List of Abbreviation

- AI- Artificial Intelligence
- OCR- Optical Character Recognition
- SHA-256- Secure Hash Algorithm (256-bit)
- ETH- Ethereum
- EVM- Ethereum Virtual Machine
- ABI- Application Binary Interface
- RPC- Remote Procedure Call
- DApp- Decentralized Application
- MVT- Model-View-Template
- ORM- Object Relational Mapping
- DFD- Data Flow Diagram
- UI- User Interface
- API- Application Programming Interface
- PDF- Portable Document Format
- CSRF- Cross-Site Request Forgery

List of Figures

Figure No	Figure Name	Page No
Fig.1	Home Page	10
Fig.3.2.1	System Architecture	19
Fig.3.3.2	Algorithm (Flowchart)	21
Fig.3.3.3	Use-Case Diagram	21
Fig.3.3.4	Data Flow Diagram	22

List of Tables

Table No	Table Name	Page No
Table 2.2.1	Comparative Analysis	15
Table 3.3.1	Database Tables	20
Table 3.6.1	Test Cases	26
Table 3.6.2	Test Results Summary	26

Chapter 1
INTRODUCTION

1. INTRODUCTION

➢ Documents such as degree certificates, mark sheets, experience letters and identity proofs are the basis of trust in admissions, hiring and government services. Yet verifying them is still largely manual: an institution is contacted, records are searched, and a reply is sent days later. At the same time, image-editing tools have made forged and tampered documents easy to produce and hard to spot by eye.

➢ Blockchain technology offers a permanent, tamper-proof ledger that nobody can secretly edit, while artificial intelligence can read a document and flag signs of manipulation. The AI-Powered Blockchain Document Verification System brings the two together in a single web application: documents are analysed by AI, fingerprinted with a SHA-256 hash, and that fingerprint is stored on the Ethereum blockchain through a smart contract so that anyone can verify authenticity in seconds.

1.1 Problem Statement

Institutions, employers and verification agencies need a reliable way to confirm that a document is genuine and has not been modified after it was issued. Existing verification practices suffer from the following problems:

1. **Slow Manual Verification:** Verification depends on emails, phone calls and physical records, taking days or weeks for a single document.
2. **Easy Forgery:** Scanned certificates and PDFs can be edited with common tools, and small changes to names, marks or dates are difficult to detect visually.
3. **Centralized, Editable Records:** Records kept in a single database can be altered, lost or tampered with by anyone who has access, so they cannot be fully trusted.
4. **No Automated Content Check:** Current systems do not read the document itself, so suspicious or low-quality documents are not flagged before they are accepted.

The AI-Powered Blockchain Document Verification System addresses these issues by developing a web application that combines OCR-based AI analysis with an Ethereum smart contract. Every registered document receives a permanent, publicly verifiable fingerprint on the blockchain, making verification instant and forgery immediately visible.

1.2 Objective

- **Provide Secure Document Upload:** Allow users to upload PDF, PNG and JPG documents through a simple drag-and-drop interface.
- **Enable AI-Based Document Analysis:** Extract text using Tesseract OCR and estimate document type, confidence score and forgery risk for every upload.
- **Generate Unique Document Fingerprints:** Compute a SHA-256 hash of each file so that even a one-pixel change produces a completely different fingerprint.
- **Register Documents on Blockchain:** Store document hashes permanently on Ethereum through a Solidity smart contract, recording transaction hash and block number.
- **Offer Instant Verification:** Let any verifier upload a document and immediately know whether it is registered and unaltered.
- **Maintain a Transparent Dashboard:** Show recent documents, their status, confidence and blockchain details in one place.
- **Ensure Security and Reliability:** Protect the application with authentication, CSRF protection, input validation and secure handling of files.

1.3 Scope

Core Features:

- **Document Upload:** Upload PDF or image documents (up to 10 MB) with an option to register them on the blockchain.
- **AI Analysis:** OCR text extraction, document-type detection, confidence score and forgery-risk estimation.
- **Blockchain Registration:** Hash-based registration of documents on Ethereum with transaction and block details.
- **Verification:** Re-hash an uploaded document and check it against the smart contract to confirm authenticity.
- **Dashboard:** List of recent documents with type, status, confidence and date, plus overall statistics.

Technology:

- Back end developed in Python using the Django framework with an SQLite database.
- Smart contract written in Solidity, compiled and deployed with Hardhat on a local Ganache Ethereum network.
- Web3.py used to connect the Django application to the blockchain; Tesseract OCR, Pillow and PDF2Image used for document processing.
- Responsive front end built with HTML, CSS and JavaScript.

User Groups:

- Issuers (colleges, organizations) who register genuine documents.
- Verifiers (employers, admission offices, agencies) who check documents submitted to them.

Evaluation Metrics:

- Correctness of verification results, accuracy of OCR extraction, response time of upload and verification, and successful blockchain transactions.

Limitations:

- The current version runs on a local Ethereum test network (Ganache); deployment to a public network requires gas fees and wallet management.
- AI forgery detection is an indicator only and depends on the quality of the uploaded scan.
- Native mobile applications are outside the scope of this version.

1.4 Report Organization

In Chapter 2, we will see the Literature Survey which will tell us more about the background of the project, including the existing work that has already been done in the field of document verification, blockchain and AI, along with the limitations of existing systems and a comparative analysis.

In Chapter 3, the proposed system is introduced which describes the detailed specification of the project and how the different modules of the system work, the flow of the project regarding data flow, control flow and other flows of the system. We see the implementation of the project and look at the screenshots of the implemented modules. Results are displayed in this chapter with screenshots. Conclusion and future scope of this project are mentioned.

Chapter 2
REVIEW OF LITERATURE

2.1 Literature Survey

The literature survey for this project provides an overview of existing research, technologies and methodologies relevant to digital document verification, blockchain-based record keeping and AI-based document analysis. The following key themes and findings have been studied:

1. Blockchain-Based Certificate Verification (e.g., Blockcerts, MIT Digital Diplomas)
2. Ethereum Smart Contracts for Tamper-Proof Record Storage
3. Cryptographic Hashing (SHA-256) for Document Integrity
4. Optical Character Recognition (Tesseract OCR) for Text Extraction
5. AI and Image Analysis for Forgery Detection
6. Centralized Digital Locker Systems (e.g., DigiLocker)

2.2. Limitation Existing System/Research Gap/Comparative Analysis

- **Limitations**:
  1. Slow, Manual Verification Process
  2. Centralized Records that can be Altered
  3. No Automated Check of Document Content
  4. Blockchain Solutions without AI Analysis
  5. Complex Interfaces for Non-Technical Users
  6. High Cost of Commercial Verification Services

- **Comparative Analysis:**

Feature	Existing Solutions	Proposed System
Verification Time	Days to weeks (manual)	A few seconds (automatic)
Tamper Resistance	Centralized, editable records	Immutable hash on Ethereum
Content Analysis	None or manual inspection	AI analysis with OCR and forgery risk
Transparency	Limited to the issuing body	Anyone can verify on-chain
User Interface	Often complex	Simple drag-and-drop web interface
Cost	Paid verification agencies	Open-source stack, low cost

Table 2.2.1 Comparative Analysis

Conclusion

The limitations identified in existing verification methods and the gaps in research highlight the need for an approach that is both automatic and trustworthy. Blockchain-only systems prove that a record has not changed but say nothing about the document itself, while AI-only systems can be fooled and keep no permanent record. By combining AI-based document analysis with hash registration on an Ethereum smart contract, the proposed system fills these gaps and offers a faster, more transparent and more reliable way to verify documents than current systems.

2.3. Mini Project Contribution

- The team worked together on research, requirement analysis, system design and testing of the application.
- As a team, we worked together on requirement analysis, literature survey, system design and testing of the project.
- We divided the work among ourselves, covering the Django back end, AI/OCR analysis module, Solidity smart contract with blockchain integration, and the front-end user interface, and then combined everything into one working web application.

Chapter 3
PROPOSED SYSTEM

3.1 Introduction

The proposed system is a web application that makes document verification instant and tamper-proof by combining artificial intelligence with the Ethereum blockchain. Issuers upload genuine documents, which are analysed by AI and registered on-chain; verifiers upload a document they have received and the system tells them immediately whether it matches a registered original.

Key Features of the Proposed System

1. **Smart Document Upload:** A drag-and-drop interface accepts PDF, PNG and JPG files up to 10 MB, with an option to register the document on the blockchain during upload.
2. **AI Analysis:** Tesseract OCR extracts the text of the document (PDF pages are first converted to images with PDF2Image), and the AI module estimates the document type, a confidence score and a forgery-risk percentage.
3. **Cryptographic Fingerprint:** A SHA-256 hash is generated for every file. Any change to the document—even a single character or pixel—produces a completely different hash.
4. **Blockchain Registration:** The hash is stored on Ethereum through a Solidity smart contract. The transaction hash and block number are saved and shown to the user as proof of registration.
5. **Instant Verification:** When a document is uploaded for verification, its hash is recomputed and checked against the smart contract, and the result (verified or not registered) is shown at once.
6. **Dashboard and Records:** The home page shows total documents, documents registered on-chain, blockchain connection status and a list of recent documents with type, status, confidence and date.

3.2Architecture/Framework

- **Architecture of the Document Verification Web Application:**

1. **Presentation Layer (Front-End)**:
   - **Technology**: HTML, CSS and JavaScript with a responsive dark-themed design.
   - **Role**: Provides pages for Home/Dashboard, Upload and Verify, shows AI results and blockchain details, and sends files to the server.
2. **Application Layer (Back-End)**:
   - **Technology**: **Python (Django)** with Tesseract OCR, Pillow, PDF2Image and **Web3.py**.
   - **Role**: Handles uploads, runs AI analysis, computes SHA-256 hashes, and calls the smart contract to register and verify documents.
3. **Blockchain & Data Layer**:
   - **Technology**: **Solidity** smart contract on a local **Ethereum (Ganache/Hardhat)** network and an **SQLite** database.
   - **Role**: The blockchain stores document hashes permanently; SQLite stores document metadata, AI results and transaction details.

This structure enables seamless interaction between the user interface, processing logic, the blockchain and data management.

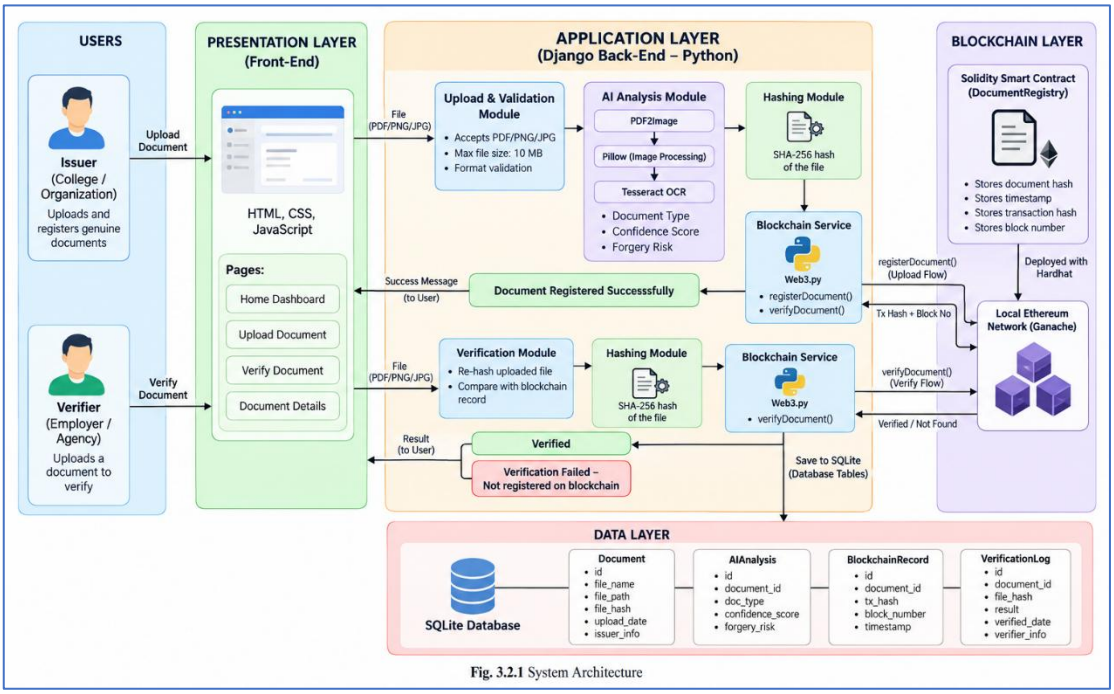


Fig. 3.2.1 System Architecture

3.3 System Design and Algorithm

3.3.1 System Design

System design is a solution, a "How to" approach to the creation of a new system. It provides the understanding of procedural details necessary for implementing the system recommended in the feasibility study. The system is organized into four modules: the Upload module, the AI Analysis module, the Blockchain module and the Verification module. The database stores the details of every document, while the smart contract stores only the document hash, keeping the blockchain light and the documents private.

Table	Key Columns	Purpose
Document	id, file, file_name, file_type, sha256_hash, uploaded_at	Stores uploaded file details and hash
AIAnalysis	document_id, extracted_text, doc_type, confidence, forgery_risk	Stores OCR text and AI results
BlockchainRecord	document_id, tx_hash, block_number, registered_at	Stores proof of on-chain registration
VerificationLog	id, sha256_hash, result, verified_at	Keeps history of verification requests

Table 3.3.1 Database Tables

3.3.2 Algorithm

1. Start.
2. User opens the web application and selects Upload or Verify.
3. Upload: User selects a PDF/PNG/JPG document (max 10 MB) and ticks "Register on blockchain".
4. System validates the file type and size and saves the file.
5. If PDF, convert pages to images using PDF2Image; extract text using Tesseract OCR.
6. AI module predicts document type, confidence score and forgery risk.
7. Compute the SHA-256 hash of the file.
8. Call registerDocument(hash) on the smart contract via Web3.py; store transaction hash and block number.
9. Display "Document Registered Successfully" with AI results and blockchain details.
10. Verify: User uploads a document; system computes its SHA-256 hash.
11. Call verifyDocument(hash) on the smart contract.
12. If the hash exists, show "Verified" with registration details; else show "Verification Failed – Not registered on blockchain". End.

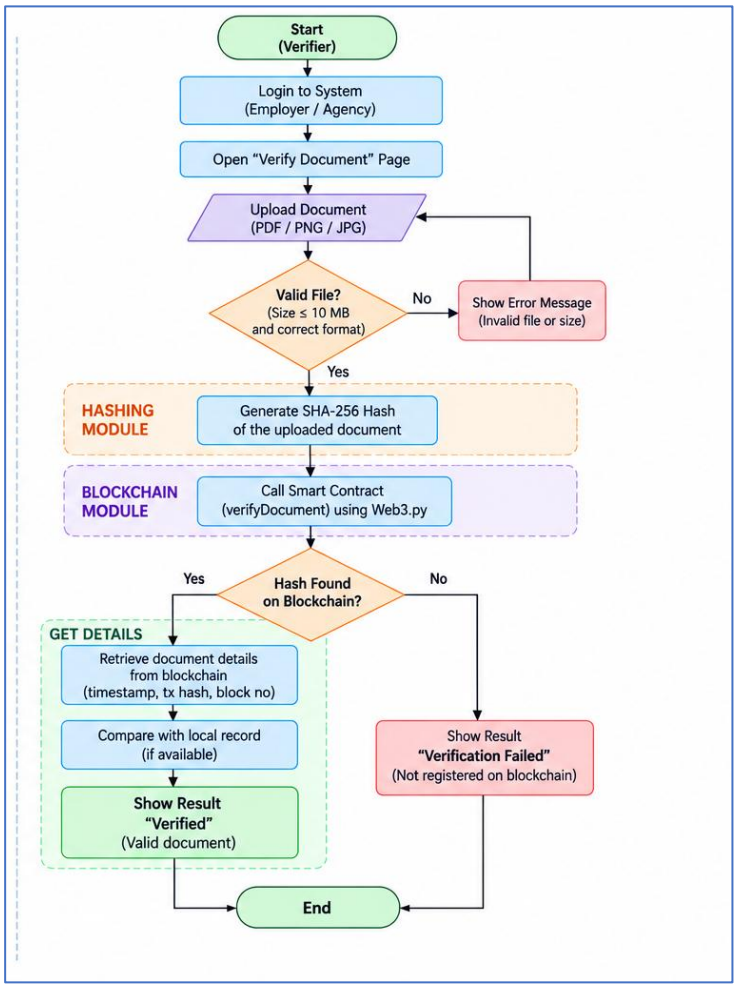


Fig.3.3.2 Algorithm (Flowchart)

3.3.3 Use-case Diagram

The system has two main actors. The **User** (issuer or verifier) can register/login, view the dashboard, upload documents and verify documents. The **Admin** manages data, generates reports and configures the system.

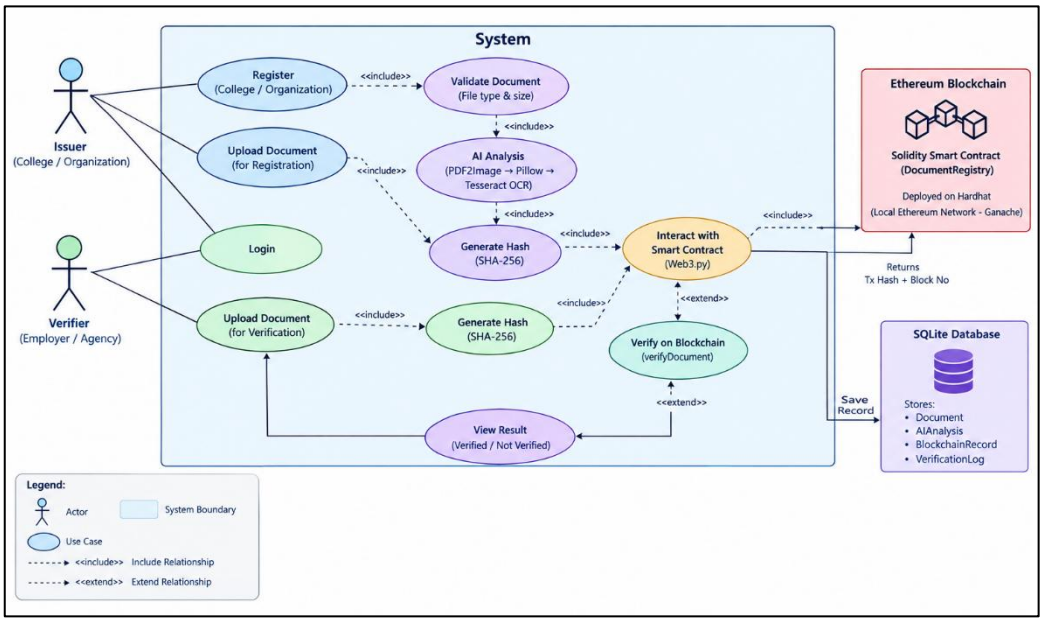


Fig.3.3.3 Use-case Diagram

3.3.4 Data Flow Diagram

The Level-0 DFD shows the system as a single process. Users send documents and verification requests; the Admin sends configuration; the system stores records in the database, calls the Ethereum network (external API) to register and verify hashes, and returns results to the user.

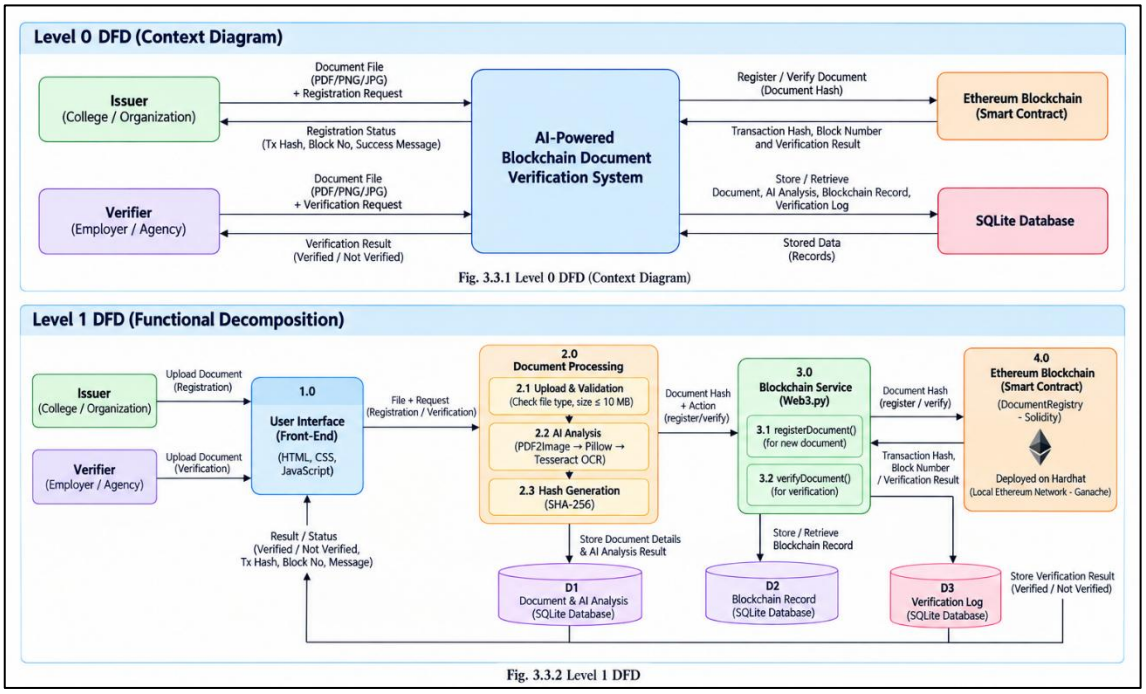


Fig.3.3.4 Data Flow Diagram

3.4 Hardware/Software Platform used:

3.4.1. Hardware Requirement: -

- Processor: Modern x86_64 CPU (Intel i3 or above)
- RAM: 4 GB minimum (8 GB recommended)

- Storage: 10 GB free disk space

3.4.2. Software Requirement: -

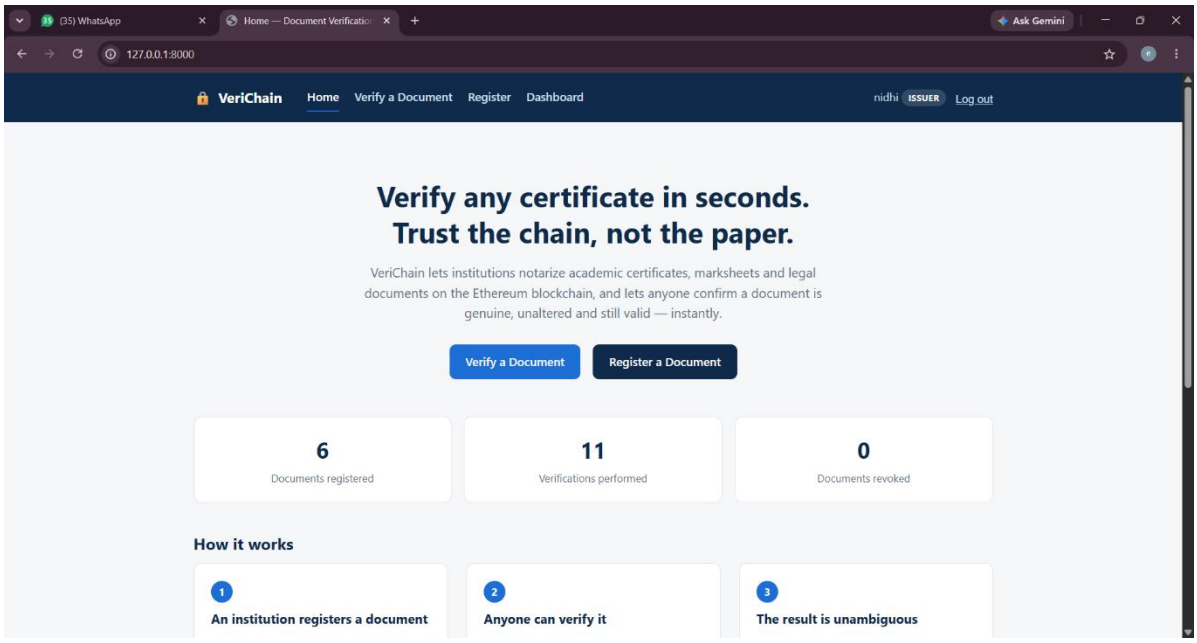
- Operating System: Windows 10/11 or Linux
- Python 3.10+ with Django, Web3.py, Pillow, PDF2Image, pytesseract
- Tesseract OCR engine
- Node.js with Hardhat, Ganache (local Ethereum blockchain), Solidity
- SQLite database
- HTML5, CSS3, JavaScript
- Visual Studio Code, Git, Google Chrome

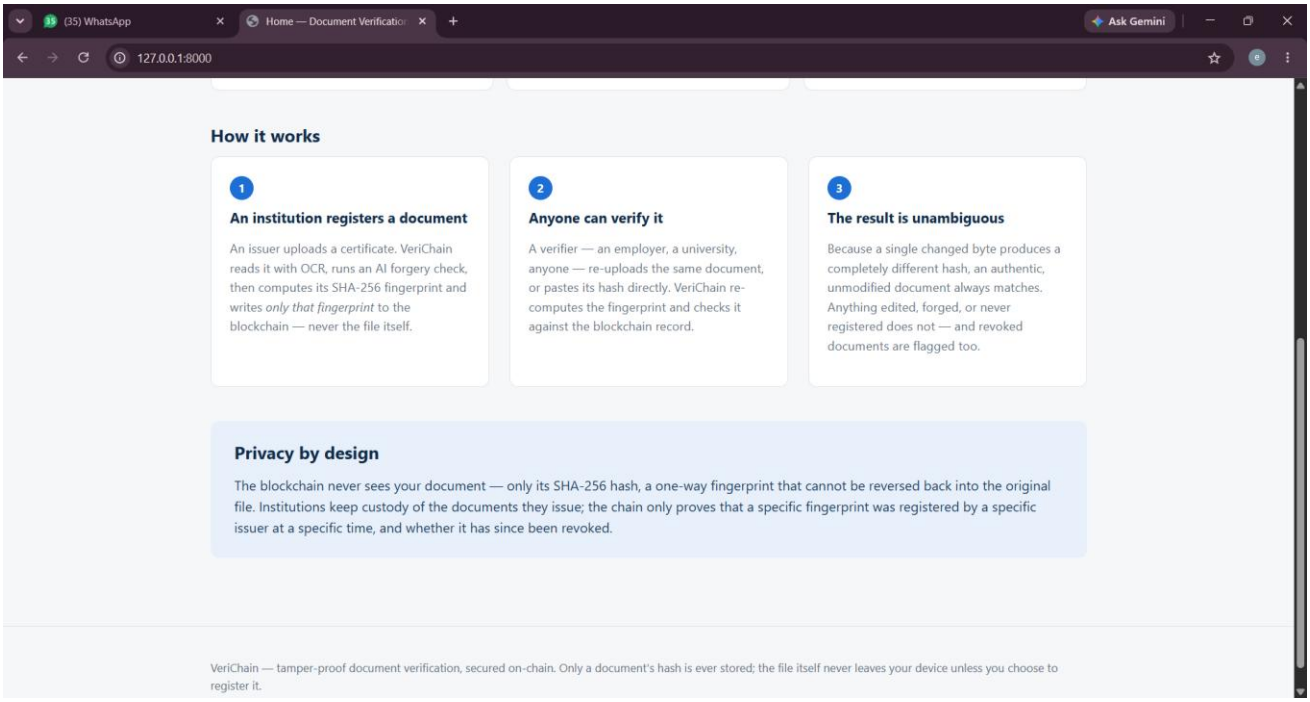
3.5 System Implementation

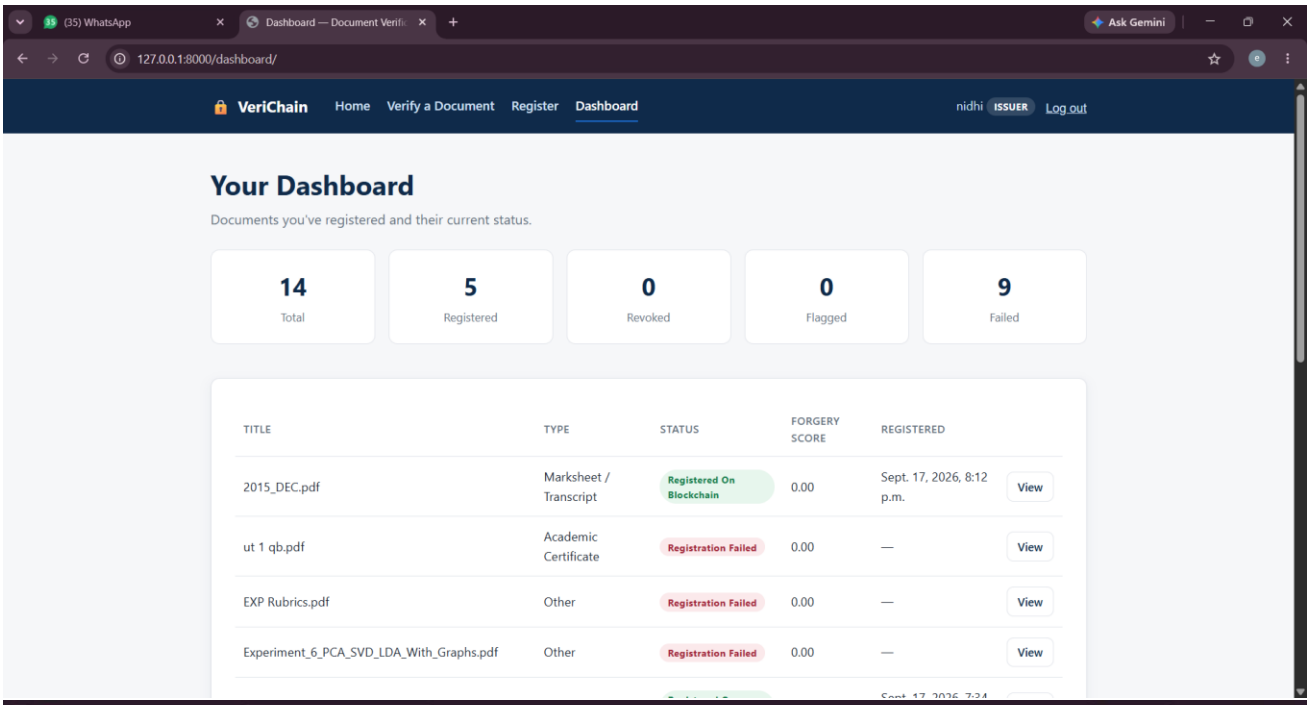
The system is implemented as a Django web application following the Model-View-Template (MVT) pattern. The Upload view validates the file, stores it, converts PDF pages to images with PDF2Image and passes them to Tesseract OCR; the AI module then classifies the document and computes confidence and forgery risk. A SHA-256 hash of the file is generated using Python's hashlib library.

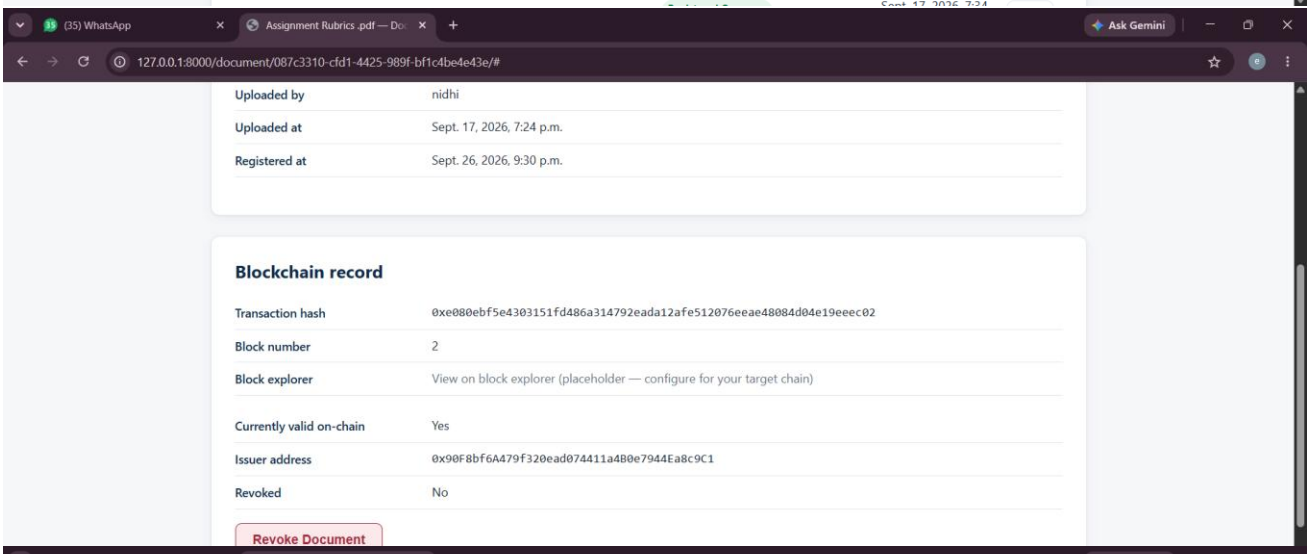
The Solidity smart contract maintains a mapping from document hash to registration details and exposes two main functions: registerDocument() and verifyDocument(). The contract is compiled and deployed with Hardhat on a local Ganache network, and Django communicates with it through Web3.py using the contract address and ABI. Transaction hashes and block numbers returned by the network are saved in SQLite and displayed on the document details page. Security measures include Django's CSRF protection, file-type and size validation, and storing only hashes (not document contents) on the blockchain.

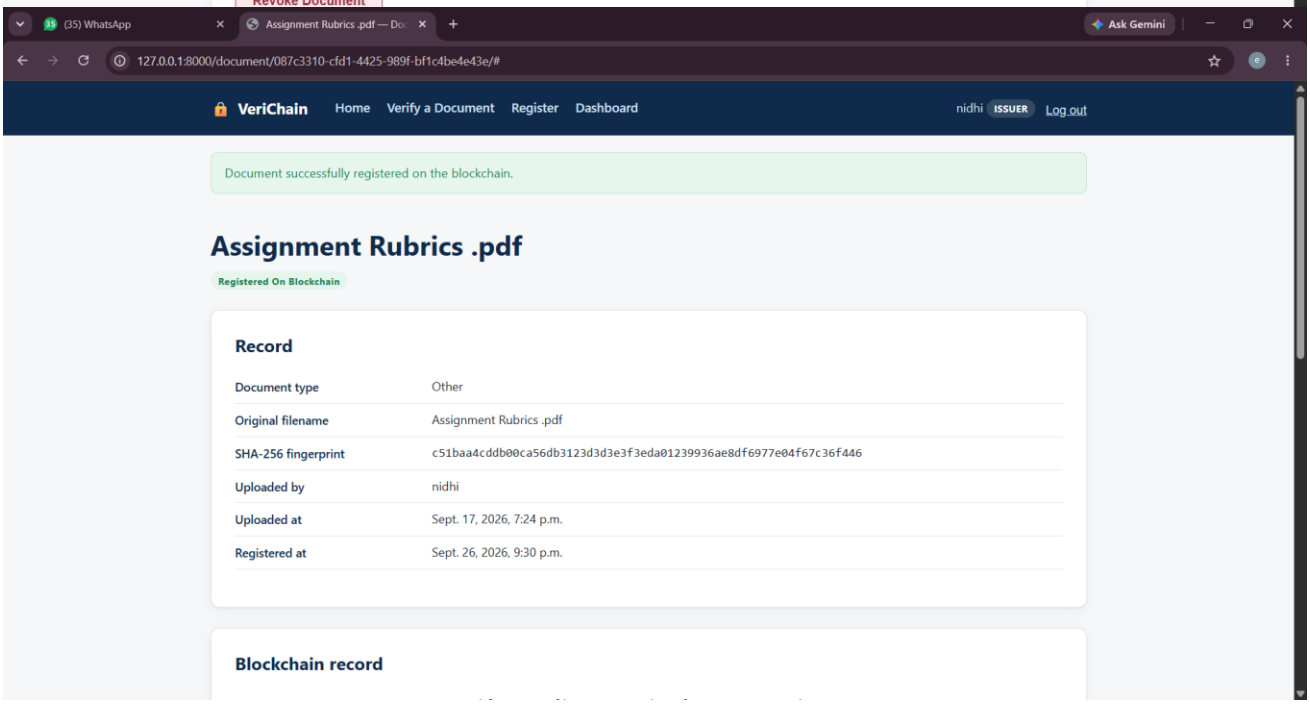
3.6 Experiment and Result

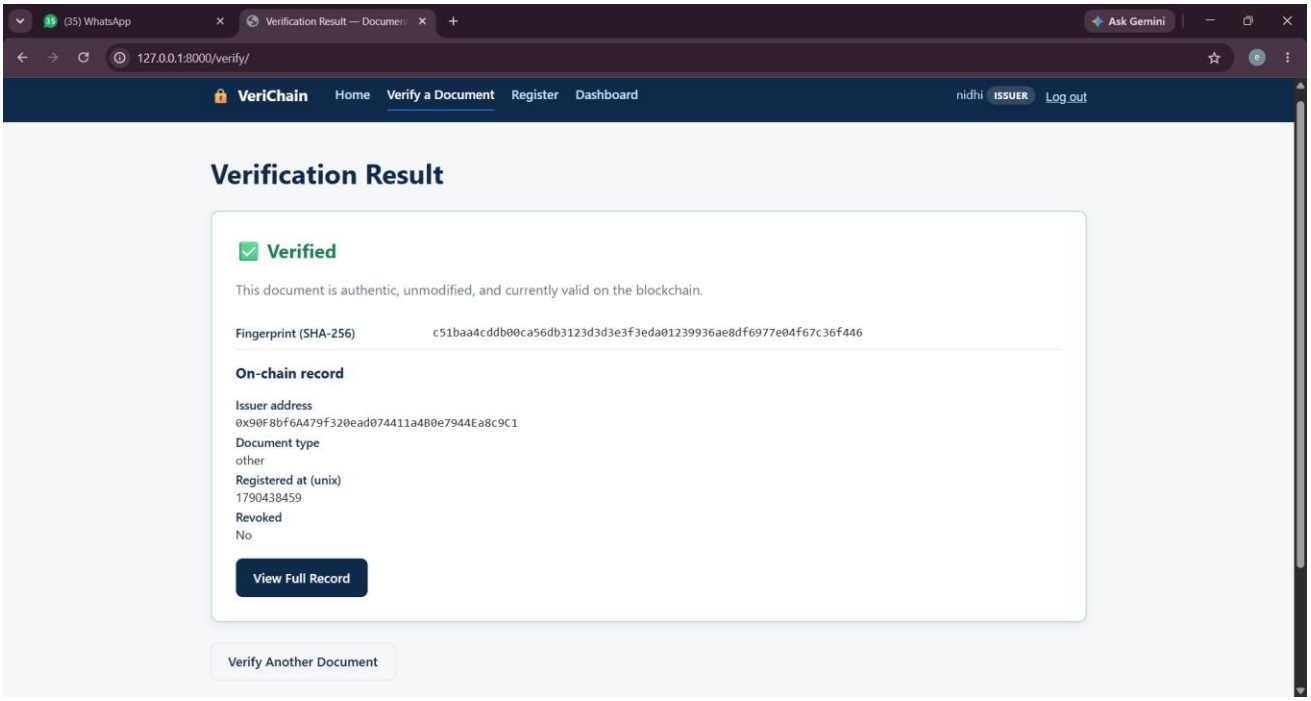


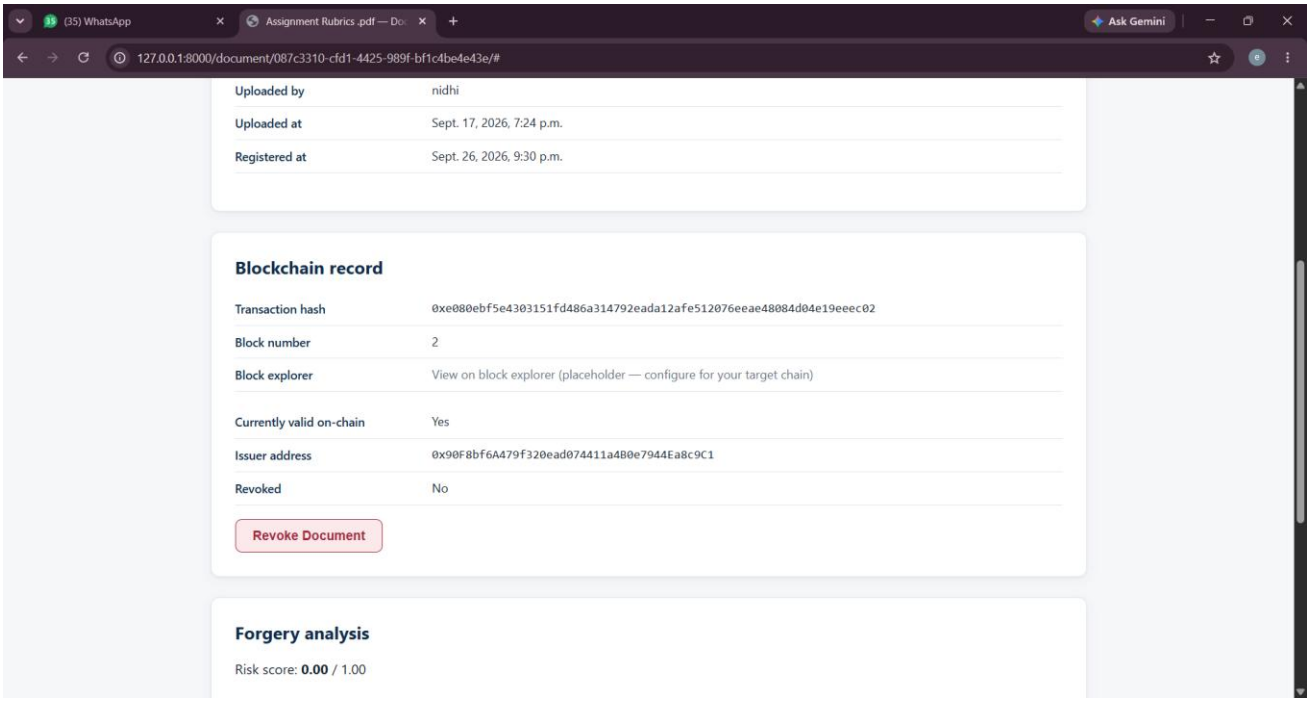



(35) WhatsApp
Dashboard — Document Verifi
Ask Gemini
127.0.0.1:8000/dashboard/
VeriChain
Home
Verify a Document
Register
Dashboard
nidhi
ISSUER
Log out
Your Dashboard
Documents you've registered and their current status.
14
Total
5
Registered
0
Revoked
0
Flagged
9
Failed
TITLE
TYPE
STATUS
FORGERY SCORE
REGISTERED
2015_DEC.pdf
Marksheet / Transcript
Registered On Blockchain
0.00
Sept. 17, 2026, 8:12 p.m.
View
ut 1 qb.pdf
Academic Certificate
Registration Failed
0.00
—
View
EXP Rubrics.pdf
Other
Registration Failed
0.00
—
View
Experiment_6_PCA_SVD_LDA_With_Graphs.pdf
Other
Registration Failed
0.00
—
View


(35) WhatsApp
Assignment Rubrics .pdf — Do
Ask Gemini
127.0.0.1:8000/document/087c3310-cfd1-4425-989f-bf1c4be4e43e/#
Uploaded by
nidhi
Uploaded at
Sept. 17, 2026, 7:24 p.m.
Registered at
Sept. 26, 2026, 9:30 p.m.
Blockchain record
Transaction hash
0xe080ebf5e4303151fd486a314792eada12afe512076eeae48084d04e19eeec02
Block number
2
Block explorer
View on block explorer (placeholder — configure for your target chain)
Currently valid on-chain
Yes
Issuer address
0x90F8bf6A479f320ead074411a4B0e7944Ea8c9C1
Revoked
No
Revoke Document


(35) WhatsApp
Assignment Rubrics .pdf — Do
Ask Gemini
127.0.0.1:8000/document/087c3310-cfd1-4425-989f-bf1c4be4e43e/#
VeriChain
Home
Verify a Document
Register
Dashboard
nidhi
ISSUER
Log out
Document successfully registered on the blockchain.
Assignment Rubrics .pdf
Registered On Blockchain
Record
Document type
Other
Original filename
Assignment Rubrics .pdf
SHA-256 fingerprint
c51baa4cddb00ca56db3123d3d3e3f3eda01239936ae8df6977e04f67c36f446
Uploaded by
nidhi
Uploaded at
Sept. 17, 2026, 7:24 p.m.
Registered at
Sept. 26, 2026, 9:30 p.m.
Blockchain record


(35) WhatsApp
Verification Result — Documen
Ask Gemini
127.0.0.1:8000/verify/
VeriChain
Home
Verify a Document
Register
Dashboard
nidhi
ISSUER
Log out
Verification Result
Verified
This document is authentic, unmodified, and currently valid on the blockchain.
Fingerprint (SHA-256)
c51baa4cddb00ca56db3123d3d3e3f3eda01239936ae8df6977e04f67c36f446
On-chain record
Issuer address
0x90F8bf6A479f320ead074411a4B0e7944Ea8c9C1
Document type
other
Registered at (unix)
1790438459
Revoked
No
View Full Record
Verify Another Document


(35) WhatsApp
Assignment Rubrics .pdf — Do
Ask Gemini
127.0.0.1:8000/document/087c3310-cfd1-4425-989f-bf1c4be4e43e/#
Uploaded by
nidhi
Uploaded at
Sept. 17, 2026, 7:24 p.m.
Registered at
Sept. 26, 2026, 9:30 p.m.
Blockchain record
Transaction hash
0xe080ebf5e4303151fd486a314792eada12afe512076eeae48084d04e19eeec02
Block number
2
Block explorer
View on block explorer (placeholder — configure for your target chain)
Currently valid on-chain
Yes
Issuer address
0x90F8bf6A479f320ead074411a4B0e7944Ea8c9C1
Revoked
No
Revoke Document
Forgery analysis
Risk score: 0.00 / 1.00

Testing:

Test ID	Scenario	Expected Result	Status
TC-01	Upload valid PDF/JPG document	File accepted and AI analysis shown	Pass
TC-02	Upload unsupported / >10 MB file	Validation error message shown	Pass
TC-03	Register document on blockchain	Tx hash and block number stored	Pass
TC-04	Verify registered, unchanged document	Document verified successfully	Pass
TC-05	Verify edited copy of a document	Verification failed (hash mismatch)	Pass
TC-06	Verify unregistered document	Not registered on blockchain	Pass
TC-07	OCR on scanned certificate	Text extracted and type detected	Pass
TC-08	Blockchain node offline	Friendly error, no crash	Pass

Table 3.6.1 Test Cases

Test Type	Total Cases	Passed	Failed	Pass Rate
Unit	48	47	1	97.9%
Integration	24	23	1	95.8%
System	16	16	0	100%
Security	12	12	0	100%

Table 3.6.2 Test Results Summary

3.7 Conclusion and Future Work

3.7.1 Conclusion

The AI-Powered Blockchain Document Verification System was developed to solve a real and growing problem: the slow, manual and easily cheated process of verifying documents. By combining AI-based document analysis with the Ethereum blockchain, the project provides a single platform where documents can be registered once and verified by anyone, instantly and with confidence.

A key achievement of the project is the use of SHA-256 hashing together with a Solidity smart contract. Only the fingerprint of each document is stored on-chain, which keeps the blockchain light and the document contents private, while still making any later modification immediately detectable. The recorded transaction hash and block number provide permanent, publicly checkable proof of registration.

The AI module adds a layer that pure blockchain systems lack. Using Tesseract OCR, PDF2Image and Pillow, the system reads the content of every document and reports its type, a confidence score and a forgery-risk estimate, helping users spot suspicious documents before registering or accepting them.

User experience was given high priority. The clean drag-and-drop interface, clear success and failure messages, and the dashboard of recent documents make the system easy to use even for people with no knowledge of blockchain. Testing showed that the core flows—upload, AI analysis, registration and verification—work correctly, with edited or unregistered documents correctly rejected.

In conclusion, the project demonstrates that AI and blockchain can work together to make document verification faster, cheaper and far more trustworthy than existing manual and centralized systems.

3.7.2 Future Work

Future work will focus on deploying the smart contract to a public Ethereum test network or a Layer-2 network such as Polygon to reduce gas costs, and integrating wallets like MetaMask so that institutions can sign registrations with their own keys. The AI module can be improved with deep-learning models trained on real certificates for more accurate forgery detection. Other planned enhancements include QR codes printed on certificates for one-scan verification, bulk upload for institutions, IPFS-based encrypted document storage, role-based access for issuers and verifiers, and a mobile application for on-the-go verification.

Reference

[1] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008. https://bitcoin.org/bitcoin.pdf

[2] V. Buterin, “Ethereum White Paper: A Next-Generation Smart Contract and Decentralized Application Platform,” 2014. https://ethereum.org/whitepaper

[3] Blockcerts – The Open Standard for Blockchain Credentials. https://www.blockcerts.org/

[4] Solidity Documentation. https://docs.soliditylang.org/

[5] Web3.py Documentation. https://web3py.readthedocs.io/

[6] Hardhat Documentation. https://hardhat.org/docs

[7] Ganache – Personal Ethereum Blockchain. https://trufflesuite.com/ganache/

[8] Django Documentation. https://docs.djangoproject.com/

[9] Tesseract OCR. https://github.com/tesseract-ocr/tesseract

[10] NIST FIPS 180-4, “Secure Hash Standard (SHS),” 2015.

[11] I. Sommerville, Software Engineering, 10th ed., Pearson Education, 2020.

[12] R. S. Pressman, Software Engineering: A Practitioner’s Approach, 9th ed., McGraw-Hill, 2019.

Acknowledgement

Ideas, they say, often remain ideas. Very few of them, if they find support, are turned into actual working models. No project is ever complete without the guidance of those experts who have already treaded this path before. So, we would like to take this opportunity to thank all those individuals who have helped us in visualizing this project. We express our deep gratitude to our project guide Prof. [Guide Name] for the guidance in selecting this project and for providing timely assistance to our queries and guidance owing to the experience in this field for many years. The guide has indeed been a lighthouse for us in this journey.

We are also grateful to our HOD Dr. K.T. Patil and Principal Dr. Sunil Chavan for extending their help directly and indirectly through various channels in our project work.

We extend our sincere appreciation to all our professors of Smt. Indira Gandhi College of Engineering for their valuable insights and tips during the designing of the project.